



Le langage de requêtes SQL avec SQL Server

Module 03 – DDL : la gestion de la structure
de la base de données

1



OBJECTIFS

Créer une table

Connaître les différents types
de données

Appliquer des contraintes pour
assurer la cohérence des
données

Comprendre le principe des
index et les utiliser à bon
escient



2

Instruction du DDL

Nature de l'objet à créer

Espace de stockage pour les objets et contraintes créés. Par défaut **dbo**

```
CREATE TABLE [nom_schema.] nom_table  
(  
    nom_colonne type_colonne  
    [, nom_colonne ... ]  
    ....  
);
```

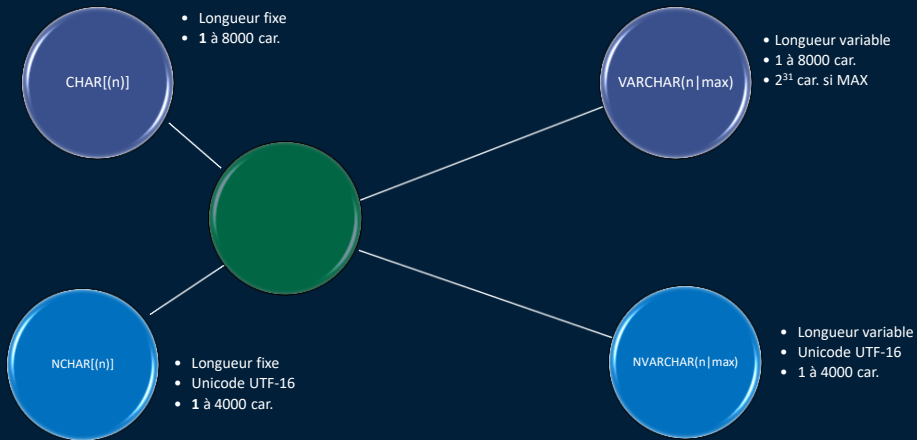
Identifiant :

- 1 à 128 caractères
- 1^{er} caractère : lettre, @, _, #
- Puis caractères alphanumériques
- Exemple :
 Gestion_employes,
 [Gestion_employes],
 "Gestion_Employes"

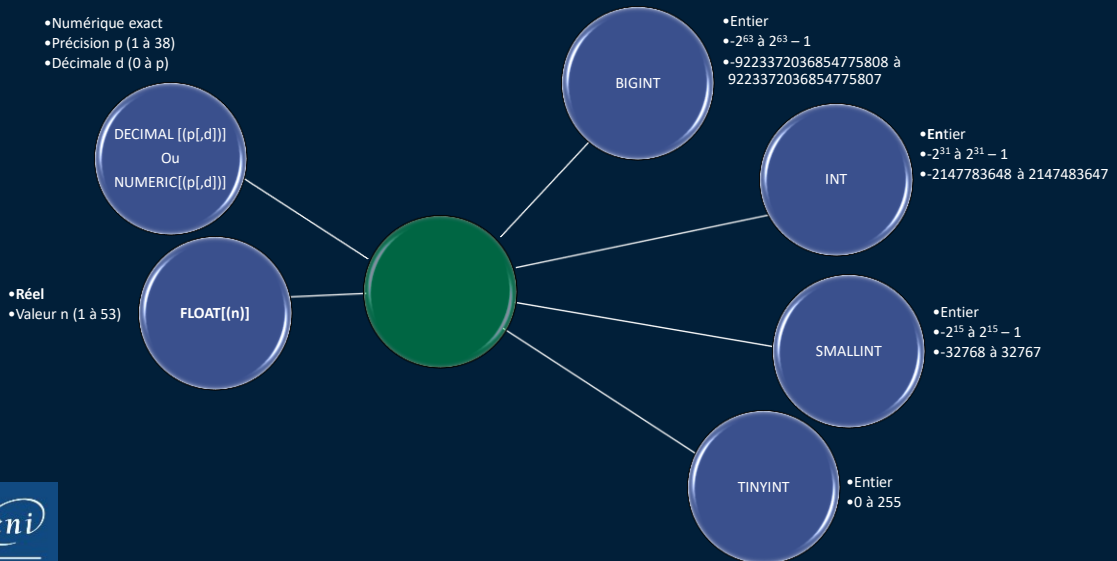


- Mots clés SQL en majuscules (exemple : CREATE TABLE)
- Nom des tables au pluriel et en snake_case (exemple : Conges_mens)
- Nom des colonnes explicite, en minuscule et en snake_case (exemple : date_modif)
- Règles de nommage cohérentes

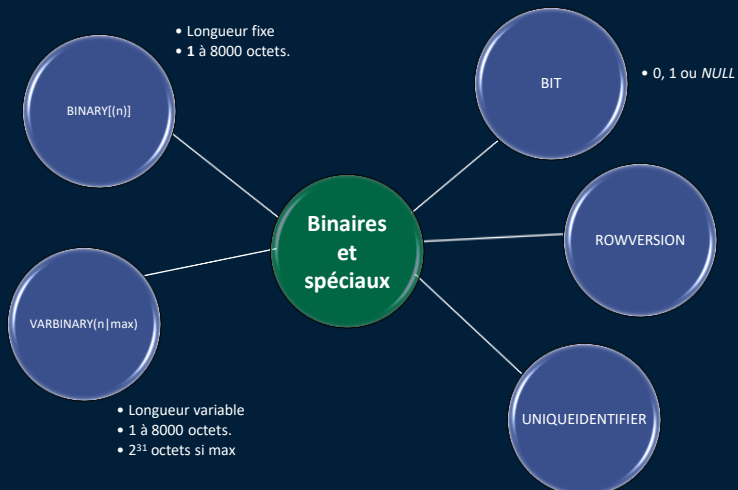
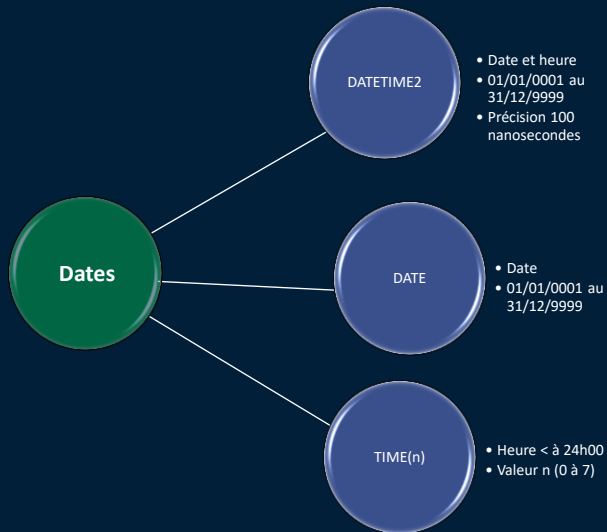




5



6



Synonyme	Type SQL Server
Caractères	
CHAR VARYING	VARCHAR
CHARACTER	CHAR(N)
CHARACTER VARYING(N)	VARCHAR(N)
NATIONAL CHARACTER(N)	NCHAR(N)
NATIONAL CHAR(N)	NCHAR(N)
NATIONAL CHARACTER VARYING(N)	NVARCHAR(N)
NATIONAL CHAR VARYING(N)	NVARCHAR(N)
NATIONAL TEXT	NTEXT
Numériques	
DEC	DECIMAL
DOUBLE PRECISION	FLOAT
INTEGER	INT
Binaires	
BINARY VARYING	VARBINARY
Autres	
ROWVERSION	TIMESTAMP



Création d'une table avec des attributs typés

DÉMONSTRATION

```
CREATE TABLE Clients(  
  id_client      INT,  
  nom            VARCHAR(50),  
  prenom        VARCHAR(40),  
  date_naissance DATE,  
  ville         VARCHAR(50),  
  portable      NUMERIC(10),  
  fixe          NUMERIC(10)  
);
```



Les types de données

**TRAVAUX
DIRIGÉS**

A partir du *MRD – GesCom.pdf*

construisez, sur le modèle de notre table Clients, les tables Commandes, Details_commandes, Articles et Categories.



11

MISE EN ŒUVRE DE L'INTÉGRITÉ DES DONNÉES

- Objectifs
 - Assurer la cohérence des données
 - Appliquer les règles de fonctionnement issues de l'analyse
 - Traduction des règles du modèle relationnel
- Mise en œuvre
 - Attribut requis (NULL / NOT NULL)
 - Contrainte (CONSTRAINT)
 - Déclencheur de base de données (TRIGGER)



12

- NULL ou NOT NULL

```
CREATE TABLE Clients(  
  id_client      INT          NOT NULL,  
  nom            VARCHAR(50)  NOT NULL,  
  prenom        VARCHAR(40)  NULL,  
  date_naissance DATE        NULL,  
  ville         VARCHAR(50)  NOT NULL,  
  portable      NUMERIC(10)  NULL,  
  fixe          NUMERIC(10)  NULL  
);
```



13

```
CREATE TABLE nom_table  
(  
  nom_colonne1 type [contrainte],  
  nom_colonne2 type [contrainte],  
  nom_colonne3 type [contrainte],  
  [contrainte],  
  [contrainte],  
  ....  
);
```

Contrainte niveau
colonne



La contrainte ne doit
porter que sur cette
colonne

Contrainte niveau
table



14

- Pas obligatoire mais conseillé !
- Syntaxe :
 CONSTRAINT nom_contrainte contrainte
- Exemple :

```
CONSTRAINT PK_Clients PRIMARY KEY
```



typeDeContrainte_nomTable[_complementInfos]

- typeDeContrainte :
 - PK : clé primaire (primary key)
 - UN ou UK : clé secondaire (unique key)
 - CK : contrainte de validation (check)
 - DF : valeur par défaut (default)
 - FK : clé étrangère (foreign key)
- nomTable : la table portant la contrainte
- complementInfos :
 - Pour PK, aucune info complémentaire nécessaire
 exemple : PK_Employes
 - Pour FK, le nom de la table référencée
 exemple : FK_Employes_Services
 - Pour UK, CK, DF, le nom de la colonne portant la contrainte
 exemple : DF_Employes_salaire



- Niveau colonne

```
CREATE TABLE Clients(  
  id_client      INT          NOT NULL  
    CONSTRAINT PK_Clients PRIMARY KEY,  
  nom            VARCHAR(50)  NOT NULL,  
  ...  
);
```

- Niveau table

```
CREATE TABLE DetailsCommandes(  
  id_commande    INT,  
  Numero_ligne   INT,  
  ...  
  CONSTRAINT PK_DetailsCommandes  
    PRIMARY KEY(id_commande, numero_ligne)  
);
```



```
CREATE TABLE Clients(  
  id_client      INT          NOT NULL CONSTRAINT PK_Clients PRIMARY KEY,  
  nom            VARCHAR(50)  NOT NULL,  
  prenom         VARCHAR(40)  NULL,  
  date_naissance DATE         NULL,  
  ville          VARCHAR(50)  NOT NULL,  
  portable       NUMERIC(10)  NULL CONSTRAINT UN_Clients_portable UNIQUE,  
  fixe           NUMERIC(10)  NULL,  
  CONSTRAINT UN_Clients_nomPrenomDateN UNIQUE(nom, prenom, date_naissance)  
);
```

Niveau colonne

Niveau table



```
CREATE TABLE Clients(  
  id_client      INT          NOT NULL CONSTRAINT PK_Clients PRIMARY KEY,  
  nom            VARCHAR(50) NOT NULL,  
  prenom         VARCHAR(40) NULL,  
  date_naissance DATE          NULL  
    CONSTRAINT CK_Clients_dateNaissance CHECK(date_naissance < GETDATE()),  
  ville          VARCHAR(50) NOT NULL,  
  portable       NUMERIC(10) NULL CONSTRAINT UN_Clients_portable UNIQUE,  
  fixe           NUMERIC(10) NULL,  
  CONSTRAINT UN_Clients_nomPrenomDateN UNIQUE(nom, prenom, date_naissance),  
  CONSTRAINT CK_Clients_telephones CHECK(portable IS NOT NULL OR fixe IS NOT NULL)  
);
```

Niveau colonne

Niveau table



```
CREATE TABLE Clients(  
  idClient INT CONSTRAINT pk_clients PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  prenom VARCHAR(40) NULL,  
  dateNaissance DATE NULL CONSTRAINT ck_clients_dateNaissance  
    CHECK(dateNaissance < GETDATE()),  
  ville VARCHAR(50) NOT NULL  
    CONSTRAINT df_clients_ville DEFAULT 'Nantes',  
  portable NUMERIC(10) NULL CONSTRAINT un_clients_portable UNIQUE,  
  fixe NUMERIC(10) NULL,  
  CONSTRAINT un_clients_nomPrenomDateN UNIQUE(nom, prenom,  
    dateNaissance),  
  CONSTRAINT ck_clients_telephones CHECK(portable IS NOT NULL OR  
    fixe IS NOT NULL)  
);
```

Niveau colonne



Les contraintes d'intégrité de données sous SQL Server

TRAVAUX DIRIGÉS



À partir du *MRD – GesCom.pdf*

appliquez les contraintes suivantes sur les tables :

- Contraintes de non nullité
- Clés primaires
- Clés uniques
 - Dans la table Catégories, un enregistrement est aussi identifié par son libelle et son identifiant de catégorie parent.
 - Dans la table Articles, un enregistrement est aussi identifié par sa désignation
- Contraintes de validation
 - La colonne statut n'accepte que les valeurs (AP, EP, PR, AC, LI, EL).
 - La colonne date_cmd n'est valide que dans le cas où elle est inférieure à la date du jour.

CLÉ ÉTRANGÈRE (SYNTAXE)

• Syntaxe

```
[FOREIGN KEY (nom_colonne, ...)] REFERENCES  
nom_table(nom_colonne, ...)
```

```
[ON DELETE {NO ACTION | CASCADE | SET NULL | SET DEFAULT}]
```

```
[ON UPDATE {NO ACTION | CASCADE | SET NULL | SET DEFAULT}];
```

- NO ACTION : la suppression/modification d'une clé primaire est impossible si elle est référencée (option par défaut).
- CASCADE : la suppression/modification d'une clé primaire se répercute sur la clé étrangère.
- SET NULL : la suppression/modification de la clé primaire valorise la clé étrangère à null.
- SET DEFAULT : la suppression/modification de la clé primaire valorise la clé étrangère à sa valeur par défaut.



- Niveau colonne

```
CREATE TABLE Commandes(  
  id_commande INT CONSTRAINT PK_Commandes PRIMARY KEY,  
  ...  
  id_client INT NOT NULL CONSTRAINT FK_Commandes_Clients  
    REFERENCES Clients(id_client)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

- Niveau table

```
CREATE TABLE Commandes(  
  id_commande INT CONSTRAINT PK_Commandes PRIMARY KEY,  
  ...  
  id_client INT NOT NULL,  
  CONSTRAINT FK_Commandes_Clients  
    FOREIGN KEY(id_client) REFERENCES Clients(id_client)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```



23

- Ajouter une colonne :

```
ALTER TABLE nom_table  
  ADD nom_colonne type [NULL | NOT NULL] [contrainte];
```

```
ALTER TABLE Articles ADD description VARCHAR(100) NOT NULL;
```

- Modifier une colonne :

```
ALTER TABLE nom_table  
  ALTER COLUMN nom_colonne type [NULL | NOT NULL];
```

```
ALTER TABLE Articles ALTER COLUMN description VARCHAR(500) NULL;
```

- Supprimer une colonne :

```
ALTER TABLE nom_table  
  DROP COLUMN nom_colonne;
```

```
ALTER TABLE Articles DROP COLUMN description;
```



24

LA MODIFICATION DES CONTRAINTES DES TABLES

- Ajouter une contrainte :

```
ALTER TABLE nom_table [WITH CHECK | WITH NOCHECK]
ADD [CONSTRAINT nom_contrainte] contrainte;
```

```
ALTER TABLE Articles
WITH NOCHECK
ADD CONSTRAINT FK_Articles_Categories
FOREIGN KEY(id_categorie)
REFERENCES Categories(id_categorie);
```

```
ALTER TABLE Articles
ADD CONSTRAINT CK_Articles_prixHt
DEFAULT 10 FOR prix_ht;
```

- Supprimer une contrainte :

```
ALTER TABLE nom_table
DROP CONSTRAINT nom_contrainte;
```

```
ALTER TABLE Clients DROP CONSTRAINT UN_Clients_portable;
```

- Activer/désactiver une contrainte (ne s'applique qu'aux contraintes de clé étrangère et de validation) :

```
ALTER TABLE nom_table
{CHECK | NOCHECK} CONSTRAINT {ALL | nom_contrainte};
```

```
ALTER TABLE Articles NOCHECK CONSTRAINT ALL;
```



Les contraintes d'intégrité référentielle sous SQL Server

TRAVAUX DIRIGÉS

À partir du *MRD – GesCom.pdf*

appliquez les contraintes suivantes sur les tables :

- Contraintes d'intégrité référentielle entre les tables Details_commandes et Categories et de Categories vers Categories.

Préparez un script de suppression de l'ensemble des contraintes d'intégrité référentielle.



Le schéma de la base de données

DÉMONSTRATION



27

LES VALEURS AUTO-INCRÉMENTÉES

- Identity :
 - Génère des valeurs entières
 - Associé à une colonne d'une table
 - Pas nécessairement mais généralement sur la clé primaire
 - 1 seul par table
 - Peut produire des trous dans la numérotation
 - Désactivable/réactivable
- Séquence :
 - Génère des valeurs entières
 - Compteur défini indépendamment de toute table
 - Peut être utilisé pour générer des valeurs pour autre chose qu'une colonne



28

- **Syntaxe :**

nom_colonne type_entier IDENTITY[(début, incr)] ...

- début : première valeur utilisée (1 par défaut)
- incr : incrément pour passer à la valeur suivante (1 par défaut)

- **Exemple :**

```
CREATE TABLE Categories (  
  id_categorie INT IDENTITY NOT NULL,  
  libelle VARCHAR(50),  
  ...  
);
```



- **Syntaxe :**

```
CREATE SEQUENCE nomSequence  
[ AS typeEntier ]  
[ START WITH valeurDeDepart ]  
[ INCREMENT BY pasIncrement ]  
[ MINVALUE valeurMini | NO MINVALUE ]  
[ MAXVALUE valeurMaxi | NO MAXVALUE ]  
[ CYCLE | NO CYCLE ]  
[ CACHE nombreValeursEnCache | NO CACHE ];
```

- **Exemple :**

```
CREATE SEQUENCE SQ_Clients AS INT START WITH 15 INCREMENT BY 5;
```



- Syntaxe :

`NEXT VALUE FOR nom_sequence`

- Exemple :

```
CREATE TABLE Clients(  
  idClient CONSTRAINT DF_Clients_idClient  
  DEFAULT NEXT VALUE FOR SQ_Clients  
  ...  
)
```



Les valeurs auto-incrémentées

TRAVAUX DIRIGÉS

À partir du *MRD – GesCom.pdf*

Mettre en place une valeur à incrémentation automatique sur la clé primaire de la table Categories.

Créer une séquence qui sera utilisée comme valeur par défaut pour valoriser la clé primaire de la table Clients.



- Syntaxe :

```
CREATE TABLE nom (  
    ...,  
    colonne AS expression_calculée [PERSISTED],  
    ...  
);
```

- Exemple :

Articles	id_article	designation	prix_ht
	45	perforateur	90,00
	22	foret SDS 12	6,90
	37	burin plat SDS	4,70

```
CREATE TABLE Articles(  
    idArticle      INT  
        CONSTRAINT pk_articles PRIMARY KEY,  
    designation    VARCHAR(50) NOT NULL  
        CONSTRAINT un_articles_designation UNIQUE,  
    prixHT         NUMERIC(8,2) NOT NULL,  
    prixTTC        AS prixHT*1.2  
);
```



- Syntaxe :

```
DROP TABLE nom_table;
```

- Exemple :

```
DROP TABLE Clients;
```



Attention, une table ne peut pas être supprimée si elle est référencée par une contrainte d'intégrité référentielle



**TRAVAUX
DIRIGÉS**

La suppression des tables

À partir du *MRD – GesCom.pdf*

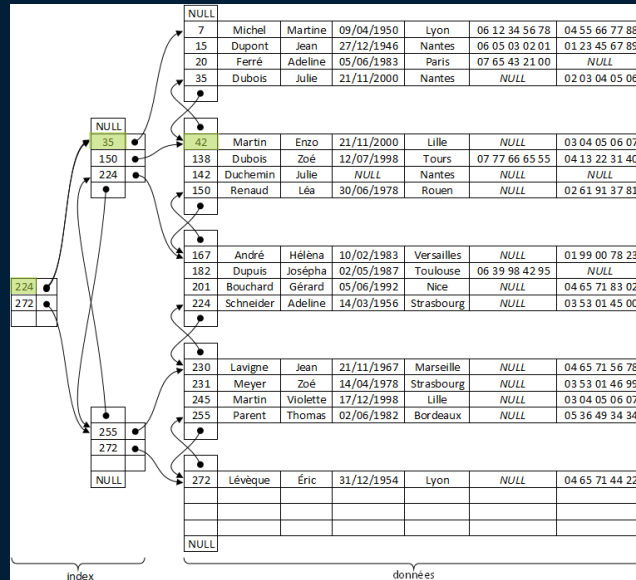
Créer un script de suppression des tables



- Objectif :
 - Accéder plus rapidement à l'information
- Catégories d'index :
 - Organisé
 - Non-organisé
- Caractéristiques d'index :
 - Unique
 - Filtré
 - Couvrant

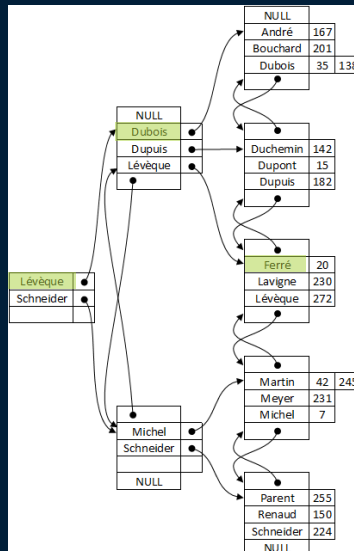


Exemple :
recherche du client n°142



37

Exemple :
recherche du client Lavigne



38

- Pas sur des index créés automatiquement
 - Clés primaires (index organisé et unique)
 - Clés secondaires (index non-organisé et unique)
- Sur les attributs utilisés pour :
 - Des jointures (clés étrangères)
 - Des restrictions
 - Des regroupements
 - Des tris



Pour les performances, il est préférable d'avoir moins d'index que trop d'index



- Syntaxe :

```
CREATE [UNIQUE] INDEX nom_index
ON {nom_table|nom_vue} (nom_colonne [ASC|DESC], ...
[INCLUDE(nom_colonne, ...)]
[WHERE restriction];
```

- Exemples :

```
CREATE INDEX IN_Commandes_idClient ON Commandes(id_client);
```

```
CREATE INDEX IN_Articles_designation ON Articles(designation)
INCLUDE(id_categorie);
```

```
CREATE INDEX IN_Clients_nomPrenom ON Clients(nom, prenom);
```

```
CREATE UNIQUE INDEX IN_Clients_portable ON Clients(portable)
WHERE portable IS NOT NULL;
```



- Syntaxe :
DROP INDEX nom_index ON nom_table;

- Exemple :

```
DROP INDEX IN_Commandes_idClient ON Commandes;
```



- Syntaxe :
ALTER INDEX {nom_index|ALL} ON nom_table {REORGANIZE|REBUILD};

- Exemples :

```
ALTER INDEX IN_Commandes_idClient ON Commandes REBUILD;
```

```
ALTER INDEX ALL ON Clients REORGANIZE;
```



Locations (1^{ère} partie) DDL

**TRAVAUX
PRATIQUES**

